

Systeme de reconnaissance d'entités nommées appliqué au wolof

Fine-tuning d'afro-xlmr-base sur le corpus MasakhaNER

Ousmane Faye Xavy Ams'elda Mengue Nek Rodolphe Guidogba

Traitement automatique des langues

July 24, 2026

Plan

- 1 Introduction
- 2 Les données
- 3 Préparation des données
- 4 Architecture du modèle
- 5 Justification des choix
- 6 Entraînement et évaluation
- 7 Limites, application et conclusion

Qu'est-ce que la reconnaissance d'entités nommées ?

La **reconnaissance d'entités nommées** (NER) identifie et catégorise, dans un texte, les groupes de mots désignant des objets du monde réel.

Exemple annoté

Macky Sall (PER) dafa dem **Ndakaaru** (LOC) ci
bésup aldiouma 17 juillet 2026 (DATE) nguir ndajé ak **njitu rewmi**, ci watiay djogalénté
ONU (ORG).

Quatre types d'entités dans notre corpus :

PER
Personnes

LOC
Lieux

DATE
Dates

ORG
Organisations

Objectif et démarche

Objectif. Construire un système capable de prédire l'étiquette d'entité de chaque mot d'un texte wolof, à partir des ensembles *train* et *dev* fournis, et l'évaluer sur *test*.

Nature de la tâche

Il s'agit d'une **classification de tokens** (une étiquette par mot), et non d'une classification de phrases. C'est ce qui détermine toute l'architecture.

Démarche. Exploration des données → préparation → fine-tuning d'un transformer pré-entraîné → évaluation au niveau entité → comparaison de modèles → application de démonstration.

Le corpus MasakhaNER Wolof

Format **CoNLL** : un token par ligne, suivi de son étiquette ; les phrases sont séparées par une ligne vide.

Schéma BIO

- B-XXX — **B**egin : début d'entité
- I-XXX — **I**nside : suite de l'entité
- O — **O**utside : hors entité

Soit 9 étiquettes possibles.

Extrait

```
SAFIYETU  B-PER
BÉÉY      I-PER
Céy       O
Koronaa   O
!         O
```

	Train	Dev	Test
Phrases	1 871	267	539
Tokens	37 243	5 294	11 682

Exploration : deux constats déterminants

1. Fort déséquilibre des classes

94 % des tokens portent l'étiquette O.



Répondre « O » partout donnerait 94 % d'exactitude et zéro utilité.

2. Beaucoup de mots inconnus

49 % du vocabulaire de *test* n'apparaît jamais dans *train*.

Un modèle travaillant sur des mots entiers serait désarmé. Il faut un modèle à **sous-mots**, capable de gérer du vocabulaire jamais vu.

Conséquence directe : évaluation au **niveau entité** + modèle à **sous-mots pré-entraîné**.

Du mot au sous-mot

Le modèle ne connaît pas les mots entiers : son vocabulaire est composé d'environ **250 000 sous-mots**. Un mot est découpé en fragments connus.

Exemple réel (tokenizer SentencePiece)

"SAFIYETU" → ["_", "SAF", "IYE", "TU"]

- Aucun mot n'est totalement inconnu : au pire il est réduit à des caractères. **C'est ce qui gère les 49 % de mots inconnus.**
- Le symbole _ marque un début de mot : il permet de recoller les morceaux.

Le problème d'alignement (point technique clé)

Le corpus fournit **une étiquette par mot** ; le modèle prédit **une étiquette par sous-mot**. Il faut redistribuer les étiquettes.

Règle retenue

- 1^{er} sous-mot → étiquette du mot
- sous-mots suivants → -100
- jetons spéciaux (<s>, </s>) → -100

-100 est la valeur *ignorée* par la fonction de coût : ces positions ne comptent ni dans l'apprentissage ni dans l'erreur. Le modèle est jugé sur exactement une prédiction par mot.

Alignement obtenu

sous-mot	word_id	label
<s>	None	-100
_	0	B-PER
SAF	0	-100
IYE	0	-100
TU	0	-100
_B	1	I-PER
É	1	-100
</s>	None	-100

Pourquoi pas tous les sous-mots ? Dupliquer B-PER sur les 4 fragments déclarerait 4 débuts d'entité pour une seule personne.

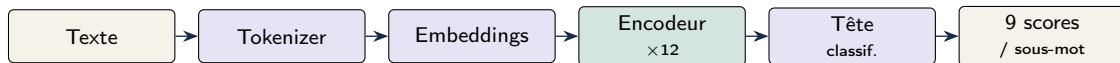
Vue d'ensemble : fine-tuning d'un transformer

Principe

Partir d'un modèle qui *comprend déjà* le texte (afro-xlmr, pré-entraîné), puis lui ajouter une tête de classification et l'ajuster sur notre tâche.

Justifié par l'exploration : moins de 2 000 phrases, c'est **trop peu pour apprendre une langue de zéro**. L'encodeur apporte la connaissance linguistique ; l'entraînement n'apprend plus qu'à repérer les entités.

Les cinq étages traversés par une phrase :



Étage 1–2 : tokenizer et embeddings

1. Tokenizer SentencePiece

Découpe le texte en sous-mots issus d'un vocabulaire de $\sim 250\,000$ entrées, appris statistiquement. Gère nativement les mots inconnus.

2. Couche d'embedding

Chaque sous-mot devient un vecteur de **768 dimensions**, somme de :

- l'**embedding de token** — l'identité du sous-mot ;
- l'**embedding de position** — sa place dans la phrase.

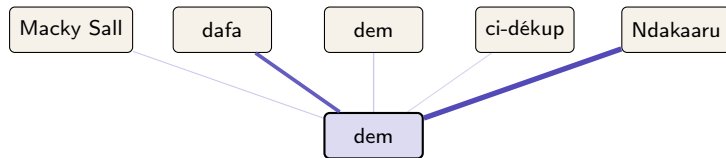
Pourquoi la position ? Le transformer voit toute la phrase d'un coup. Sans encodage de position, « Usmaan gis na Móodu » et « Móodu gis na Usmaan » seraient identiques pour lui.

Étage 3 : l'encodeur — l'auto-attention

Cœur du modèle : transforme le vecteur « brut » de chaque sous-mot en un vecteur **contextualisé**. Pour chaque token, on calcule une requête Q , une clé K et une valeur V .

Formule de l'attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$



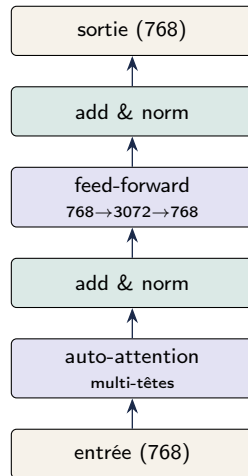
Épaisseur = poids d'attention accordé par « dem ».

Attention bidirectionnelle : chaque token regarde à gauche et à droite — exactement ce dont le NER a besoin.

Étage 3 : détail d'un bloc d'encodeur

12 blocs identiques empilés, chacun avec deux sous-couches :

- **Multi-têtes (12 têtes)** : l'attention faite 12 fois en parallèle, chaque tête pouvant se spécialiser (syntaxe, proximité. . .).
- **Feed-forward** : $768 \rightarrow 3072 \rightarrow 768$, transforme l'information dans chaque token.
- **Résiduel + normalisation** autour de chaque sous-couche, pour un entraînement stable sur 12 couches.



Étage 4 : la tête de classification

La seule partie neuve

Une simple couche linéaire projette chaque vecteur de 768 dimensions vers **9 scores**, un par étiquette. Un softmax donne des probabilités, l'argmax donne l'étiquette prédite.

C'est ce qui explique l'avertissement au chargement : `some weights were not initialized... you should TRAIN this model`. Ces poids-là étaient aléatoires.

Limite structurelle (piste d'amélioration)

Chaque token est classé *indépendamment* : rien n'interdit un I-PER après un O. Le modèle l'évite statistiquement, sans garantie. $\Rightarrow$ Une couche **CRF** modéliserait les transitions.

Chiffres clés et pré-entraînement

Blocs d'encodeur	12
Dimension cachée	768
Têtes d'attention	12
Dim. feed-forward	3 072
Vocabulaire	~250 000
Longueur max	512
Paramètres	~270 M

Comment il a appris

Pré-entraînement par **masquage de mots** : on cache 15 % des tokens, le modèle les devine d'après le contexte. Aucune annotation humaine → des téraoctets de texte brut.

C'est cette connaissance que le fine-tuning réutilise : on n'apprend pas la langue au modèle, seulement à repérer les entités.

Pourquoi afro-xlmr — et le cas du wolof

MAFT : Multilingual Adaptive Fine-Tuning (Alabi et al., 2022)

afro-xlmr = XLM-R adapté par pré-entraînement supplémentaire sur **17 langues africaines** + arabe, français, anglais. **Architecture identique à XLM-R ; seuls les poids changent.**

Honnêteté scientifique : le wolof n'est pas dans afro-xlmr

Ni dans les 100 langues de XLM-R, ni dans les 17 d'afro-xlmr. Le modèle fait donc du **transfert cross-lingue**, grâce à :

- l'écriture latine partagée (sous-mots communs) ;
- la proximité typologique avec d'autres langues africaines vues ;
- le français, massivement mêlé au wolof écrit.

⇒ Notre score reflète une *généralisation* vers une langue jamais vue, pas une simple mémorisation.

Pourquoi le F1 au niveau entité

Les trois mesures

- **Précision** : parmi les entités annoncées, combien sont correctes ?
- **Rappel** : parmi les entités réelles, combien sont trouvées ?
- **F1** : moyenne harmonique des deux.

$$F_1 = 2 \cdot \frac{\text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}}$$

Niveau entité, pas token

Une entité n'est juste que si **tous** ses tokens sont corrects (bonnes frontières, bon type). Calcul via `seqeval`.

Pourquoi ? Avec 94 % de 0, l'exactitude est trompeuse. Le F1 entité ne peut être élevé que si précision et rappel le sont — la mesure de référence sur données déséquilibrées.

Paramètres d'entraînement

Paramètre	Valeur	Raison
Taux d'apprentissage	2×10^{-5}	ajuster sans effacer le pré-entraînement
Taille de lot	16	compromis stabilité / mémoire GPU
Époques (max)	15	plafond ; l'arrêt anticipé agit avant
Décroissance des poids	0,01	régularisation (petit corpus)
Patience	3	arrêt si le F1 dev stagne

Arrêt anticipé et sélection du meilleur modèle

L'entraînement s'est arrêté à l'époque 8 : le F1 de validation ne progressait plus.

`load_best_model_at_end` conserve le modèle de la **meilleure** époque, pas de la dernière. Continuer aurait produit du **surapprentissage**.

Résultats sur le test

Global (539 phrases)

F1 = 0,625

Précision 0,611 Rappel 0,641

L'ordre des types suit exactement leur **fréquence** dans le corpus — ce qui était *prévu* dès l'exploration.

Type	Préc.	Rappel	F1	Occ.
LOC	0,769	0,820	0,794	211
PER	0,569	0,653	0,609	176
ORG	0,351	0,491	0,409	55
DATE	0,394	0,186	0,252	70

Analyse des erreurs. La majorité sont des entités classées 0 (sous-détection), pas des confusions de type. DATE souffre le plus : rare et longue (3,2 tokens en moyenne).

Comparaison de modèles

Protocole

Même pipeline, mêmes données, mêmes hyperparamètres, même early stopping. **Seul le modèle de base change** — toute différence de F1 est donc attribuable au pré-entraînement.

	afro-xlmr-base	xlm-roberta-base
Pré-entraînement africain	oui (17 langues)	non
Architecture	identique (12 couches, 768 dim.)	
F1 global (test)	0.625	0.605

On s'attend à un avantage pour afro-xlmr (il « connaît » d'autres langues africaines et gère mieux les mots inconnus), mais **c'est la mesure qui tranche**. Reporter ici les deux F1 obtenus.

Limites et pistes d'amélioration

Limites connues

- **DATE** largement manquée (rappel 0,19).
- **ORG** instable (55 occurrences en test).
- Sous-détection > confusion de type.
- Domaine journalistique étroit.

Améliorations

- Pondérer la perte (classes rares).
- Augmenter les données DATE / ORG.
- Passer à afro-xlmr-large.
- Ajouter une couche CRF.
- Assouplir l'arrêt anticipé.

Usage responsable

Prototype académique : les entités extraites, en particulier les dates et organisations, demandent une vérification humaine.

Application de démonstration (Streamlit)

Interface « Gara » en quatre sections :

- **Annoter** : texte surligné, tableau des détections, export CSV / JSON, seuil de confiance réglable.
- **Performances** : scores réels et analyse des erreurs.
- **Documentation** : tout le processus expliqué.
- **Limites** : faiblesses et améliorations.

Cœur technique

Réutilise le pipeline HuggingFace avec `aggregation_strategy="simple"`, qui regroupe les sous-mots en entités lisibles.

Conclusion

- Système de NER wolof par **fine-tuning d'afro-xlmr-base**, **F1 = 0,625** au niveau entité sur le test.
- Chaque choix technique découle de l'**exploration des données** : déséquilibre → F1 entité ; mots inconnus → modèle à sous-mots ; petit corpus → fine-tuning.
- Compréhension complète de l'architecture : sous-mots, attention bidirectionnelle, tête de classification, transfert cross-lingue vers une langue jamais vue.
- **Portée.** Le même pipeline s'étend aux 9 autres langues de MasakhaNER — un débouché concret pour le Sénégal et la sous-région.

Références

 J. O. Alabi, D. I. Adelani, M. Mosbach, D. Klakow.

Adapting Pre-trained Language Models to African Languages via Multilingual Adaptive Fine-Tuning.

COLING 2022, p. 4336–4349.

aclanthology.org/2022.coling-1.382

 A. Conneau et al.

Unsupervised Cross-lingual Representation Learning at Scale (XLM-R).

ACL 2020. arxiv.org/abs/1911.02116

 D. I. Adelani et al.

MasakhaNER: Named Entity Recognition for African Languages.

TACL 2021. arxiv.org/abs/2103.11811

 A. Vaswani et al.

Attention Is All You Need.

NeurIPS 2017. arxiv.org/abs/1706.03762

 H. Nakayama.

sequeval: A Python framework for sequence labeling evaluation. 2018.

Merci de votre attention

Questions ?